Multi-Agent Reinforcement Learning for Cooperative Robot Navigation

AI and Robotics Laboratory Project Report

Student Information

Student: Denny Irafasha **Matriculation Number**: VR 527935
Course: AI and Robotics Laboratory
Project Topic: Multi-Agent Reinforcement Learning for Cooperative Robot Navigation

---

Abstract

Autonomous robot navigation is a fundamental problem in artificial intelligence and robotics. Robots operating in shared environments must be capable of reaching assigned goals while avoiding obstacles and coordinating their actions with other agents. This project investigates the application of Reinforcement Learning (RL), specifically the Q-Learning algorithm, to solve a cooperative navigation problem involving multiple autonomous robots.

A 5×5 grid-world environment was developed containing static obstacles, goal locations, and two autonomous agents. The robots learned navigation policies through trial-and-error interactions with the environment using a reward-based learning mechanism. The project was first implemented as a single-agent system and later extended to a multi-agent setting where two robots learned simultaneously while avoiding collisions. Experimental results demonstrate that the proposed approach successfully learned efficient navigation policies, enabling both robots to reach their destinations while avoiding obstacles and minimizing collisions.

Keywords: Reinforcement Learning, Q-Learning, Multi-Agent Systems, Robot Navigation, Artificial Intelligence.

---

1. Introduction

1.1 Background

Autonomous robots are increasingly being deployed in various sectors including manufacturing, warehouse automation, agriculture, healthcare, environmental monitoring, and transportation. One of the primary challenges faced by these systems is autonomous navigation, which requires robots to move efficiently through an environment while avoiding obstacles and reaching predefined destinations.

Traditional navigation methods such as graph search algorithms and path-planning techniques often require complete knowledge of the environment. In contrast, Reinforcement Learning (RL) enables robots to learn navigation strategies through interaction with their surroundings without requiring prior environmental knowledge.

RL has gained significant attention due to its ability to learn optimal decision-making policies from experience. This makes it particularly suitable for robotics applications where environments may be uncertain, dynamic, or partially observable.

1.2 Problem Statement

The objective of this project is to design and implement a Multi-Agent Reinforcement Learning system capable of navigating autonomous robots in a shared environment. The robots must:

- Reach assigned goal locations.
- Avoid static obstacles.
- Avoid collisions with other robots.
- Learn optimal navigation policies autonomously.

The challenge lies in enabling multiple robots to operate simultaneously while maintaining safe and efficient navigation behavior.

---

2. Project Objectives

2.1 Main Objective

To develop a Multi-Agent Reinforcement Learning framework for cooperative robot navigation using the Q-Learning algorithm.

2.2 Specific Objectives

- Design a grid-world simulation environment.
- Implement Q-Learning for autonomous navigation.
- Develop obstacle avoidance mechanisms.
- Extend the system to support multiple agents.
- Implement collision avoidance between robots.

- Evaluate the effectiveness of learned navigation policies.
- Visualize learning performance and robot trajectories.

---

3. Literature Review

3.1 Reinforcement Learning

Reinforcement Learning is a branch of machine learning in which an agent learns by interacting with an environment. The agent receives rewards or penalties based on its actions and attempts to maximize cumulative rewards over time.

The RL framework consists of:

- Agent
- Environment
- State
- Action
- Reward

Through repeated interactions, the agent gradually learns a policy that maps states to optimal actions.

3.2 Q-Learning

Q-Learning is a model-free reinforcement learning algorithm introduced by Watkins and Dayan (1992). The algorithm estimates the expected utility of taking a particular action in a given state.

The Q-value update equation is:

$Q(s,a)=Q(s,a)+\alpha[r+\gamma \max_{a'} Q(s',a')-Q(s,a)]$ Q(s,a) = Q(s,a) + \alpha \left[r + \gamma \max Q(s',a') - Q(s,a)\right]Q(s,a)=Q(s,a)+α[r+γmaxQ(s′,a′)−Q(s,a)]

Where:

Symbol	Meaning
Q(s,a)Q(s,a)Q(s,a)	Current Q-value
α\alphaα	Learning rate
γ\gammaγ	Discount factor
rrr	Reward
s′s's′	Next state

Q-Learning is widely used in robotics because of its simplicity and effectiveness in discrete environments.

3.3 Multi-Agent Reinforcement Learning

Multi-Agent Reinforcement Learning (MARL) extends RL to environments containing multiple learning agents. Each agent must learn not only from the environment but also from the actions of other agents.

Applications include:

- Autonomous vehicles
- Drone swarms
- Warehouse robots
- Traffic control systems
- Collaborative robotics

---

4. Methodology

4.1 Environment Design

A grid-world environment was created to simulate robot navigation.

Environment Parameters

Parameter	Value
Grid Size	5 × 5
Number of Robots	2
Number of Goals	2
Number of Obstacles	3
Episodes	1000

Initial Positions

Agent	Position
Robot 1	[0,0]
Robot 2	[0,1]

Goal Positions

Goal	Position
Goal 1	[4,4]
Goal 2	[4,3]

Obstacle Locations

Obstacle
[1,1]
[2,2]
[3,3]

4.2 State Space

The state of an agent is represented by its position within the grid.

For the multi-agent system, the state representation also includes information about the position of the other robot.

Example:

(Robot Position, Other Robot Position)

4.3 Action Space

Each robot can perform four possible actions:

Action	Description
Up	Move upward
Down	Move downward
Left	Move left
Right	Move right

4.4 Reward Structure

The reward function was designed to encourage efficient navigation and discourage unsafe behavior.

Event	Reward
Normal Movement	-1
Goal Reached	+100
Obstacle Collision	-20
Robot Collision	-50

This reward structure guides the learning process toward safe and optimal navigation.

4.5 Hyperparameters

The following hyperparameters were used during training:

Parameter	Value
Learning Rate (α)	0.1
Discount Factor (γ)	0.9
Exploration Rate (ε)	0.2
Training Episodes	1000

---

5. System Implementation

5.1 Single-Agent System

The project initially focused on a single robot navigating toward a goal location while avoiding obstacles.

Implementation steps:

1. Create environment.
2. Define reward structure.
3. Train agent using Q-Learning.
4. Evaluate learned policy.
5. Visualize navigation path.

This baseline implementation verified that the learning algorithm was functioning correctly before extending the system.

5.2 Multi-Agent Extension

The environment was expanded to support two independent learning agents.

Each robot maintained its own Q-table:

Q1 → Robot 1
Q2 → Robot 2

The agents learned simultaneously while sharing the same environment.

Additional mechanisms included:

- Collision detection.

- Collision penalties.
- Goal-specific rewards.
- Cooperative navigation behavior.

---

6. Results and Analysis

6.1 Single-Agent Performance

The learning curve demonstrated rapid improvement during the early training episodes.

Observations:

- Initial rewards were highly negative due to random exploration.
- Rewards increased steadily as the agent learned.
- The learned policy successfully reached the goal while avoiding obstacles.

Evaluation Results

Metric	Result
Goal Reached	True
Steps	8
Collisions	0

The learned path was:

[0,0]
[1,0]
[2,0]
[3,0]
[3,1]
[3,2]
[4,2]
[4,3]
[4,4]

6.2 Multi-Agent Performance

The multi-agent system demonstrated successful coordination between both robots.

Observations:

- Rewards increased during training.
- Both agents learned stable navigation policies.
- Obstacles were consistently avoided.

- Robot collisions were minimized.

Evaluation Results

Metric	Result
Robot 1 Goal Reached	Yes
Robot 2 Goal Reached	Yes
Obstacle Avoidance	Successful
Collision Avoidance	Successful
Mission Completion	Successful

Learning Curve Analysis

The reward curves show convergence toward higher values over time, indicating successful learning.

Moving average plots further demonstrate stable policy development for both agents.

6.3 Path Visualization

Greedy policy evaluation confirmed that:

- Robot 1 successfully reached Goal 1.
- Robot 2 successfully reached Goal 2.
- Obstacles were avoided.
- No collisions occurred during evaluation.

These results demonstrate that the learned policies generalized successfully from training to evaluation.

---

7. Discussion

The project demonstrates the effectiveness of Q-Learning in solving navigation problems within structured environments.

Strengths

- Simple and interpretable implementation.
- Successful obstacle avoidance.
- Successful multi-agent coordination.
- Effective reward-based learning.

- Clear visualization of learning behavior.

Limitations

- Small state space.
- Static obstacles only.
- Tabular Q-Learning does not scale well to larger environments.
- No communication between agents.

Despite these limitations, the project successfully achieved all objectives and provided valuable insights into Multi-Agent Reinforcement Learning.

---

8. Conclusion

This project presented a Multi-Agent Reinforcement Learning framework for cooperative robot navigation using the Q-Learning algorithm.

A grid-world environment containing obstacles and multiple agents was developed. The robots learned navigation policies through trial-and-error interactions using a reward-based learning strategy. Experimental results demonstrated successful obstacle avoidance, collision avoidance, and goal-directed navigation.

The project confirms that Reinforcement Learning can effectively solve cooperative navigation tasks and provides a foundation for more advanced Multi-Agent Reinforcement Learning research.

---

9. Future Work

Potential improvements include:

- Deep Q-Networks (DQN)
- Proximal Policy Optimization (PPO)
- Multi-Agent PPO (MAPPO)
- Dynamic obstacle environments
- Robot-to-robot communication
- Larger state spaces
- Continuous navigation environments
- Real robot implementation

References

1. Watkins, C. J. C. H., & Dayan, P. (1992). Q-Learning. *Machine Learning*, 8(3–4), 279–292.
2. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd Edition). MIT Press.
3. Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th Edition). Pearson.
4. OpenAI Gym Documentation.
5. BenchMARL Documentation.